



# VIVEKANANDA GLOBAL UNIVERSITY, JAIPUR



(Established by Act11/2012ofRajasthanGovt.Coveredu/s2(f)ofUGCAct,1956)

## Lecture Notes

Course Code - UGCSE201      Course Name - Data Structures

### Module 2: Stacks, Queues and Lists

#### Contents

- Module 2. 1. Definition, Array based implementation of stacks
- Module 2. 2. Examples- Infix, postfix, prefix representation
- Module 2. 3. Applications- Mathematical expression Evaluation
- Module 2. 4. Queues-Array based implementation of Queues
- Module 2. 5. Circular implementation of Queues, Double ended queues.
- Module 2. 6. Linked List- Singly linked Lists and Operations associated with it
- Module 2. 7. Stack and Queue Implementation Using linked list**
- Module 2. 8. Straight circular implementation of doubly linked Lists, Priority queues

#### 2.7.1 Stack Implementation Using linked list

The fact that the stack implemented with an array only functions for a predetermined number of data values is the primary issue. This implies that the quantity of data must be specified prior to implementation.

When we do not know the size of the data that will be used, an array-based stack implementation is inappropriate. A linked list data structure can be used to create a stack data structure. The linked list-based stack can be utilized with any number of values. This indicates that a linked list-based stack can handle data of varying sizes. Therefore, setting the size at the start of the implementation is unnecessary. The Stack carried out utilizing connected rundown can arrange however many information values as we need.

To carry out a stack utilizing the singly connected list, all the singly connected list tasks ought to be performed in view of Stack tasks LIFO(last in first out) and with the assistance of that information, we will execute a stack utilizing an independently connected list.

In order to implement a stack, we must therefore adhere to a straightforward rule of last in, first out, and all operations can be carried out with the assistance of a top variable. Let's investigate the Pop, Push, Peek, and Display operations with the help of following example.

### Stack Operations:

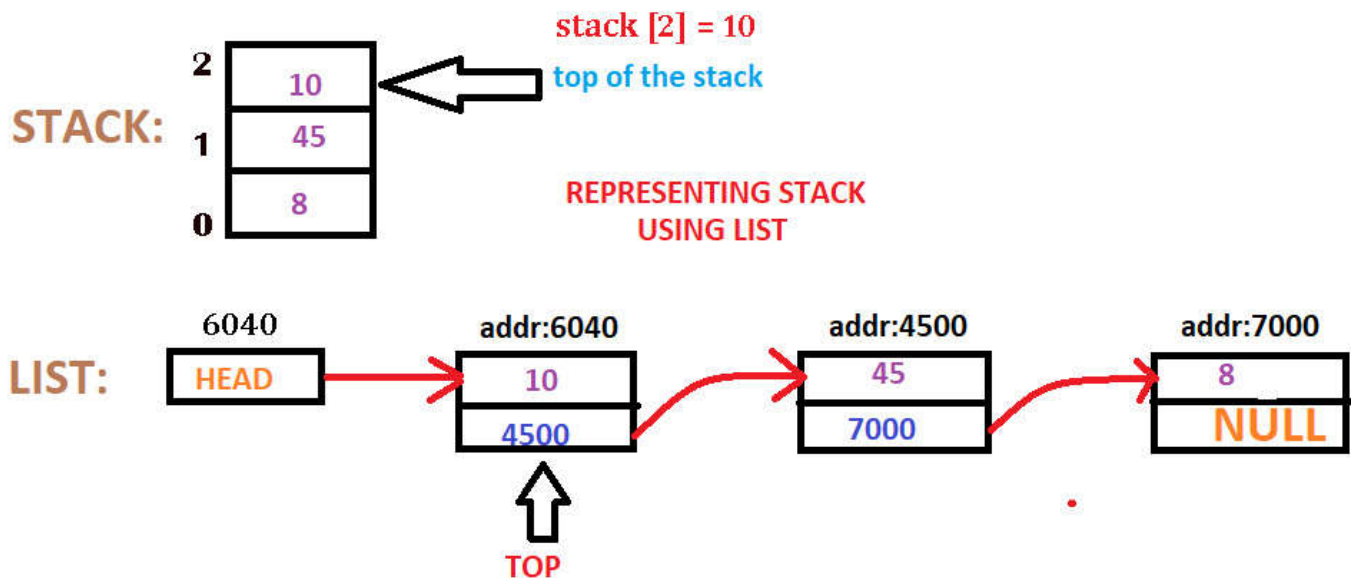
1. **Push ()**: Insert a new element into the stack i.e just insert a new element at the beginning of the linked list.
2. **Traverse or display ()**: Print all elements in Stack.
3. **Peek ()**: Return the top element.
4. **Pop ()**: Return the top element of the Stack i.e simply delete the first element from the linked list.

In linked list implementation of stack, the nodes are maintained non-contiguously in the memory. Each node contains a pointer to its immediate successor node in the stack. Stack is said to be overflow if the space left in the memory heap is not enough to create a node.

The figure below depicts a linked list representation of Stack.

Three element 8, 45, 10 are inserted respectively; thus leading 10 as a top most element. First Element inserted act as last element which contains NULL means this is the last note of the list.

In List representation just see 10 is top most element means the address of 10 is stored in special pointer variable called as START or HEAD. As we can see each node has two parts; first part contains the data itself and second part contains the address of next element.



## Overflow and underflow

Overflow occurs when there is no more space to accommodate additional elements in a data structure. In the case of a linked list, an overflow situation typically arises when trying to insert a new node when the available memory is exhausted. However, unlike arrays, linked lists dynamically allocate memory for each node, so the concept of overflow is less common.

Underflow happens when an operation is attempted on an empty data structure, such as trying to retrieve an element from an empty linked list. In the case of a linked list, an underflow situation occurs when trying to remove a node from an empty list or accessing the first/last node when the list is empty.

### 2.7.2 Stack Push() Operation Implementation : Adding node to the stack

Adding an element into the stack refers to add node to the stack; as push operation. Pushing an element to a stack in linked list implementation is different from that of an array

Implementation. In order to push an element onto the stack, the following steps are involved.

- I. Create a node first and allocate memory to it.
- II. If the list is empty then the item is to be pushed as the start node of the list. This includes assigning value to the data part of the node and assign null to the address part of the node.
- III. If there are some nodes in the list already, then we have to add the new element in the beginning of the list (to not violate the property of the stack). For this purpose, assign the address of the starting element to the address field of the new node and make the new node, the starting node of the list.

## I. Creating a node

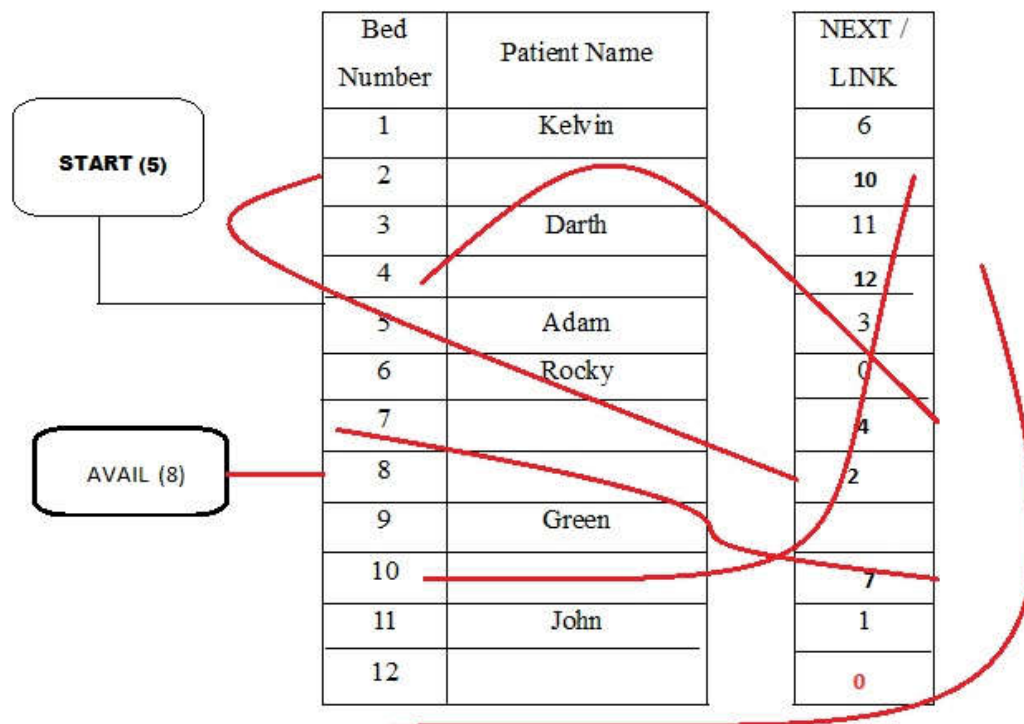
**Before going into the insertion of node; we must understand how a node can be created and maintained.** The maintenance of linked lists in memory assumes the possibility of inserting new nodes into the lists and hence requires some mechanism which provides unused memory space for the new nodes.

Analogously, some mechanism is required whereby the memory space of deleted nodes becomes available for future use.

Together with the linked lists in memory, a special list is maintained which consists of unused memory cells. This list, which has its own pointer, is called the list of available space or the free-storage list or the free pool.

Suppose our linked lists are implemented by parallel arrays as and suppose insertions and deletions are to be performed on our linked lists. Then the unused memory cells in the arrays will also be linked together to form a linked list using AVAIL as its list pointer variable. (Hence this free-storage list will also be called the AVAIL list.) Such a data structure will frequently be denoted by writing

**LIST (INFO, LINK, START, AVAIL)**



The Concept of AVAIL can be easily understand by following

First free node available can be called as AVAIL. Means address of first free memory location is stored in AVAIL. Further as we have seen in list; same procedure is applied in the AVAIL list. And last node of AVAIL list ; here it is 12 node contains NULL; which denotes the after that no free memory cells are available.

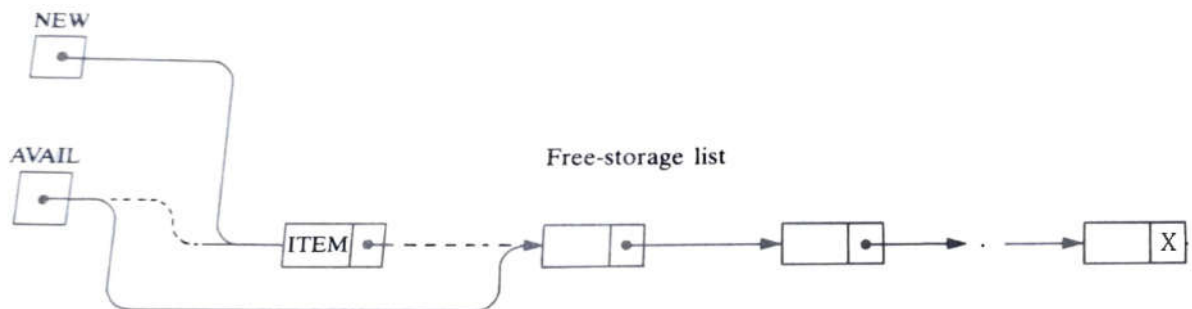
Algorithms which insert nodes into linked lists come up in various situations. We discuss only one of the three of here as we are only concentrating on push operation of stack using link list. This means adding or inserts a node at the beginning of the list. Algorithms assume that the linked list is in memory in the form LIST(INFO, LINK, START, AVAIL) and that the variable ITEM contains the new information to be added to the list. In insertion we will always do the following steps.

(a) Checking to see if space is available in the AVAIL list. If not, that is, if AVAIL = NULL, then the algorithm will print the message OVERFLOW.

(b) Removing the first node from the AVAIL list. Using the variable NEW to keep track of the location of the new node, this step can be implemented by the pair of assignments means;

**NEW:= AVAIL                      AVAIL:= LINK [AVAIL]**

We can see here is that first take first node from avail list i.e. address of first available memory and assign it to a new node called as NEW. Second after taking one node from free cell; we need to attach next free cell to AVAIL so that next we look for free memory cell, we can again look of AVAIL. That's why we kept AVAIL:= LINK [AVAIL] means now AVAIL will point next available memory.



(c) Copying new information into the new node. In other words,

**INFO[NEW]:= ITEM**

Now this summarizes creation of new node; whenever we want to insert any element in the list we have perform the above step as mandatory steps.

## II. Adding newly created node as the first element of the list



# VIVEKANANDA GLOBAL UNIVERSITY, JAIPUR

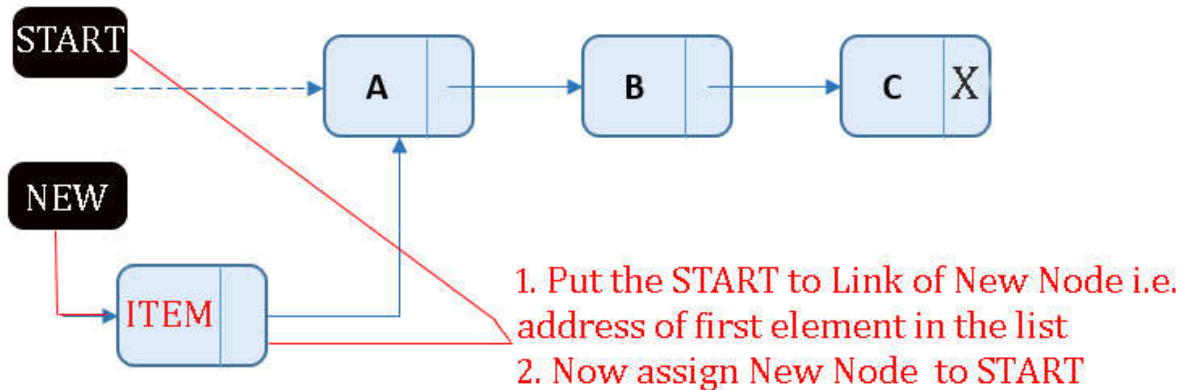


(Established by Act11/2012ofRajasthanGovt.Coveredu/s2(f)ofUGCAct,1956)

---

INSBEG(INFO, LINK, START, AVAIL, ITEM) This algorithm inserts ITEM as the first node in the list.

- Step 1 [OVERFLOW?] If AVAIL = NULL, then: Print: OVERFLOW, and Exit.
- Step 2 [Remove first node from AVAIL list.]  
**Set NEW:= AVAIL and AVAIL:= LINK[AVAIL].**
- Step 3 Set INFO[NEW]:= ITEM. [Copies new data into new node.]
- Step 4 Set LINK[NEW]:= START. [New node now points to original first node.]
- Step 5 Set START:= NEW. [Changes START so it points to the new node.]
- Step 6 Exit.



This figure shows an example of insertion any ITEM as the first element in the list.

### 2.7.3 Traverse Stack or display all elements of Stack using Link List

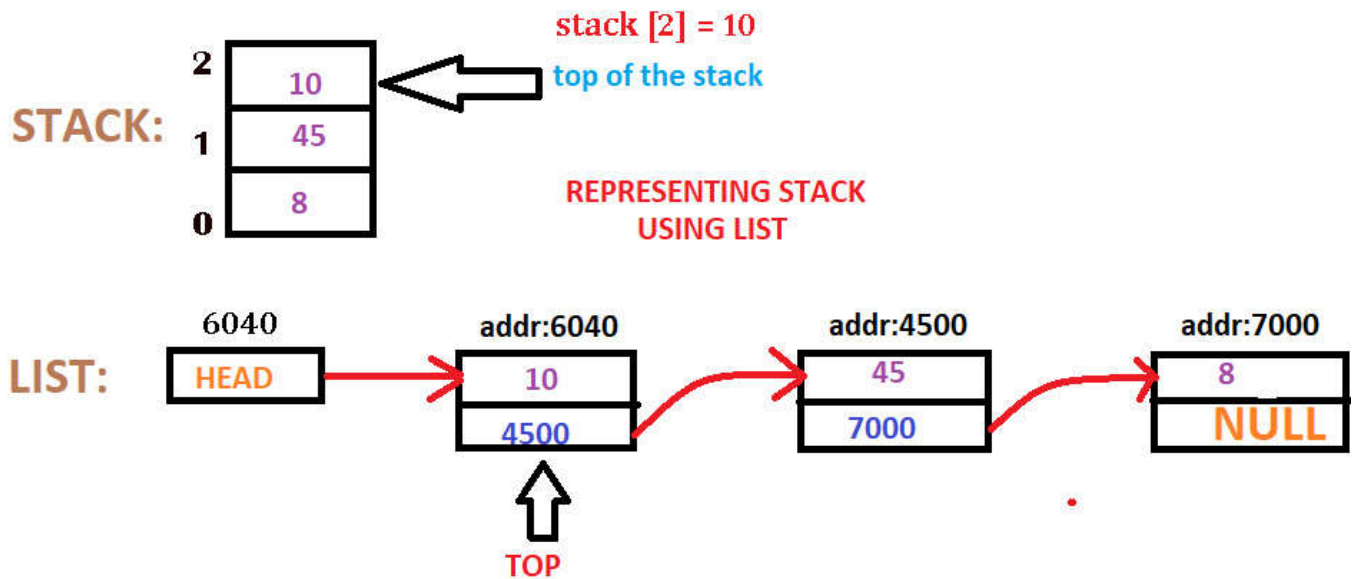
(Traversing a Linked List) Let LIST be a linked list in memory. This algorithm traverses LIST, applying an operation PROCESS to each element of LIST. The variable PTR points to the node currently being processed.

- Step 1 Set PTR START. [Initializes pointer PTR.]
- Step 2 Repeat Steps 3 and 4 while PTR  $\neq$  NULL.
- Step 3 Apply PROCESS to INFO [PTR].
- Step 4 Set PTR:= LINK[PTR] [ points to the next node to be processed ]

[End of Step 2 Loop]

- Step 5 Exit

This same process can be applied to traverse the stack using link list.



Now see how traversing is happening; list is shown with three elements 8, 45, 10 and 10 being the top most element of the stack; As seen in the picture, Now check algorithm

Step 1 Set PTR START. [Initializes pointer PTR.]

Here we are declaring a variable PTR and assign it the address of first element of the list .i.e address of 10; Now

**PTR: = 6040**

Step 2 Repeat Steps 3 and 4 while PTR  $\neq$  NULL.

[Loop until PTR reaches to NULL]

Step 3 PROCESS to INFO [PTR] i.e. Print INFO[PTR]

{ 10 value at 6040, then 45 value at 4500, then 8 value at 7000 }

Step 4 Set PTR:= LINK[PTR] [ points to the next node to be processed ]

{means now PTR contains 4500, going in loop, PTR reaches 7000, going in loop, PTR reaches NULL and loop terminates}

[End of Step 2 Loop]

Step 5 Exit

This is how traversing of stack is done with the help of Link List.

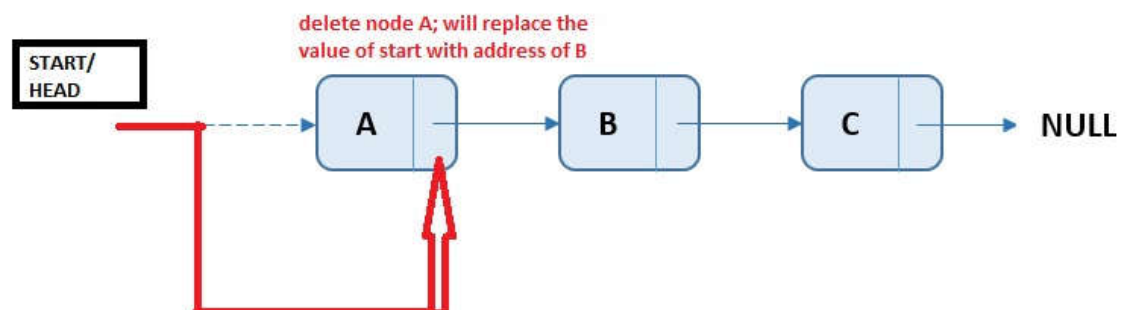
#### 2.7.4 Stack Peek() Operation Implementation : display top element

Display top element is quite easy in the link list as seen in the traverse the list;

### PROCESS to INFO [START] i.e. Print INFO[START]

#### 2.7.5 Stack Pop() Operation Implementation : remove top element

Removing the first element from the list is quite easy in the link list as seen in the following picture. Just replace the address of next element and place into START.



So pop operation will look like;

**START := LINK[START]**



# VIVEKANANDA GLOBAL UNIVERSITY, JAIPUR



(Established by Act11/2012ofRajasthanGovt.Coveredu/s2(f)ofUGCAct,1956)

---

If you want this top element to be stored for any further use; than before writing above statement; you can save the top element into the variable called as ITEM as

**ITEM = INFO [START]**

## 2.7.2 Queue Implementation Using linked list